Evaluating Classification Systems

From Model Scores to Defensible Embedded Decisions

ECE 116 Course Textbook, Chapter 2

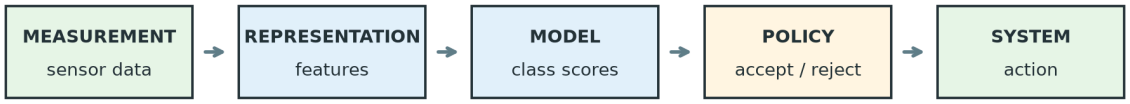
Revision 1 | August 2026

Contents

Learning objectives
How to use this chapter
1. The model is a component, not the whole system
2. Scores, argmax, and rejection
3. Begin evaluation with consequences
4. Ground truth and evaluation data
5. Confusion matrices preserve the structure of mistakes
6. Precision, recall, specificity, and F1
7. Class imbalance and deployment prevalence
8. Thresholds and operating points
9. Sequential predictions require temporal policy
10. Latency, throughput, memory, and power
11. Worked case: automated quality screening
12. Designing a defensible evaluation protocol
End-of-chapter problems
Selected responses to checks and problems
References and further reading

A trained classifier can be useful without being understood internally. It can also be misused while every line of integration code is technically correct. The difference is usually not a more sophisticated model. It is an engineer who can define the decision, measure the relevant errors, choose an operating point, and connect repeated predictions to system behavior.

This chapter develops that engineering practice. The examples include automated part inspection, equipment alarms, access requests, message triage, and streaming event detection. Each example separates the model from the policy around it so that the methods transfer across audio, image, and sensor-classification systems.



Evaluation asks where errors enter, which errors matter, and whether the final behavior is acceptable.

A classifier is one component in a larger measurement-to-action system.

Figure 1. A model produces scores. A policy interprets those scores, and the larger system determines what happens next. Evaluation must cover the complete chain.

Learning objectives

After working through this chapter, you should be able to:

1. Treat a trained model as a component with an input contract and an output contract.
2. Distinguish a score vector, an argmax label, an accepted decision, and a system action.
3. Translate the consequences of errors into measurable evaluation questions.
4. Design a held-out evaluation set with useful ground truth and independent test units.
5. Build and interpret a multiclass confusion matrix.
6. Compute and explain accuracy, precision, recall, specificity, and F1 score.
7. Explain why class balance and deployment prevalence change the meaning of metrics.
8. Sweep a confidence threshold and select an operating point from a requirement.
9. Use rejection and an unknown class without confusing the two.
10. Convert a stream of overlapping predictions into discrete events.
11. Measure latency, throughput, memory, and relative power as system properties.
12. Write an evaluation claim that is supported by a reproducible protocol.

How to use this chapter

The chapter alternates between explanation, worked examples, and retrieval checks. When you reach a check, answer it before reading the responses at the end. For every evaluation result, practice asking five questions:

- **Decision:** What exact decision rule produced this result?
- **Population:** What future inputs is the result supposed to represent?
- **Unit:** What counts as one independent test case?
- **Error:** Which mistakes are included, and which are hidden?
- **Evidence:** Could another engineer reproduce the claim from the recorded artifacts?

No single metric answers all five questions. Evaluation is the process of connecting them.

---

1. The model is a component, not the whole system

1.1 A useful black-box view

Suppose a camera observes a manufactured part. A trained model receives a processed image and returns three scores:

```
normal = 0.04
surface_defect = 0.91
dimension_defect = 0.05
```

For integration work, it is reasonable to treat the model as a black box:

```
scores = model(input)
```

The black-box view does not mean that the model is magical or exempt from engineering scrutiny. It means that the integration contract can be tested without opening the model. The contract includes at least:

- the expected input shape, units, sample rate, color order, or preprocessing;
- the output label order and score range;
- the time and memory required for one inference;
- the data distribution on which performance claims were measured;
- known limitations and conditions under which the result should not be trusted.

An ordinary software component can be wrong because of a coding defect. A learned component can also be wrong because a new input differs from its training experience. Correct integration therefore requires both conventional software tests and empirical evaluation.

1.2 Five boundaries where behavior can change

Figure 1 shows five boundaries:

1. **Measurement:** A microphone, camera, or sensor acquires physical evidence.

2. **Representation:** Preprocessing converts that evidence into the form expected by the model.
3. **Model:** The classifier maps the representation to scores.
4. **Policy:** Software accepts, rejects, delays, or combines predictions.
5. **System:** An actuator, display, log, or human workflow responds.

The same model can produce different system behavior under different policies. A part inspector that accepts every argmax label will route more parts automatically. The same inspector with a strict threshold will route fewer parts but may send ambiguous cases to a human. Neither policy changes the model weights.

1.3 Separate four terms that are often collapsed

Use precise vocabulary:

- A **score vector** is the complete numerical output from one inference.
- A **predicted label** is usually the class with the largest score.
- An **accepted decision** is a prediction that satisfies the surrounding policy.
- An **action** is the system response to an accepted decision.

The distinctions matter. A model may predict `surface_defect`, the policy may reject that prediction as uncertain, and the conveyor may therefore take no automatic action.

> **Common misconception:** "The model triggered the actuator." In a well-structured system, the model produced evidence. Application policy decided whether that evidence justified an action.

Check 1

1. Name two requirements that belong to a model's input contract.
2. A model's top class is `approve`, but its score is below the application's threshold. What are the predicted label, accepted decision, and action?
3. Why can two products using the same model have different false-alarm rates?

---

2. Scores, argmax, and rejection

2.1 What a score vector says

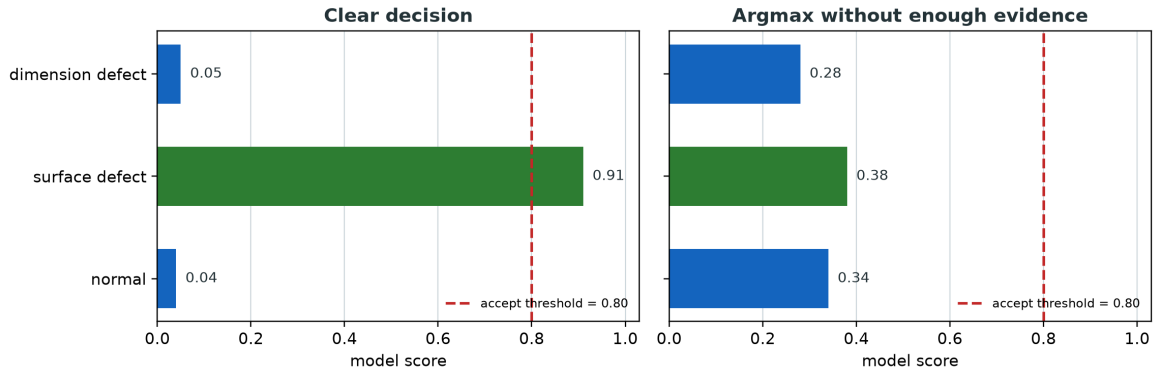
A multiclass classifier returns one score per class. Many models use a final softmax layer, so the scores are nonnegative and sum to approximately one. Even then, a score of `0.90` does not automatically mean "90 percent chance of being correct." That interpretation requires **calibration**, discussed later in this section.

The safest initial interpretation is comparative: within this inference, larger scores are more compatible with the model's learned representation than smaller scores.

For scores `s[0]` through `s[K−1]`, the argmax prediction is the index with the largest value:

```
predicted_index = max(range(len(scores)), key=lambda i: scores[i])
predicted_score = scores[predicted_index]
```

Argmax always returns a class, even when the scores are nearly tied or uniformly low. That property is convenient for benchmarking and dangerous for automatic action.



Two score vectors can have an argmax while supporting very different decisions.

Figure 2. Both examples have an argmax. Only the first satisfies the illustrated acceptance requirement.

2.2 A decision rule adds an abstain option

A simple acceptance policy is:

```
accept the argmax class if its score >= T
otherwise reject the inference
```

The threshold `T` creates a fourth outcome in the three-class inspection example: `rejected`. Rejection is not a model class. It is a policy result that means "the available evidence does not justify an automatic class decision under this rule."

Rejection can be valuable when there is a safe fallback, such as requesting another image, asking a person to inspect a part, or leaving a machine in its current state. Rejection can also be harmful if every rejected case stops a production line. Its value depends on system consequences.

2.3 Unknown class and rejection solve different problems

An **unknown class** is learned during training. It tells the model that some inputs should be represented as a named catch-all category. A **rejection rule** is imposed after training. It declines to trust a particular score vector.

These mechanisms are complementary:

Situation	Unknown class	Rejection policy
familiar non-target examples represented in training	useful	may still reject weak cases
ambiguous input between two known classes	may not help	directly useful
completely novel deployment condition	may fail unpredictably	may reduce some actions, but is not a guarantee

An unknown class cannot contain every future non-target input. A threshold cannot prove that a high-scoring unfamiliar input is safe. Neither mechanism is a universal detector for "anything the model has never seen."

2.4 Scores are not necessarily calibrated probabilities

A classifier is calibrated if, among predictions assigned a score near `0.80`, roughly 80 percent are correct. Calibration is an empirical property, not a consequence of printing two decimal places or applying softmax.

A reliability check groups predictions into score bands and compares mean score with observed correctness. For example:

score band	predictions	correct	observed accuracy
0.50 to 0.59	84	47	0.56
0.60 to 0.69	103	65	0.63
0.70 to 0.79	96	69	0.72
0.80 to 0.89	77	58	0.75
0.90 to 1.00	140	119	0.85

This model is overconfident in the upper bands: the stated score is higher than the observed fraction correct. Its scores can still rank examples well and support a useful threshold. They should not be described as literal probabilities without qualification.

Check 2

1. Why does argmax return a label for an ambiguous input?
2. What is the difference between an unknown class and a rejected inference?
3. What evidence would support interpreting a score of `0.80` as an 80 percent probability of correctness?

---

3. Begin evaluation with consequences

3.1 A metric is meaningful only relative to a decision

Consider an equipment monitor that predicts `normal` or `fault`. There are two error types:

- a **false alarm** stops or interrupts healthy equipment;
- a **miss** allows faulty equipment to continue operating.

The two errors are not interchangeable. Their costs may differ by orders of magnitude. Before selecting a metric, write the consequence of each outcome:

truth	decision	operational consequence
normal	normal	continue operating correctly
normal	fault	unnecessary inspection or downtime
fault	normal	delayed maintenance or possible damage
fault	fault	fault is caught and handled

The table does not yet specify a model threshold. It specifies what the system is trying to protect.

3.2 Turn consequences into requirements

Good requirements state a population, a metric, a bound, and often a condition:

> On held-out recordings from operators not represented in development data,
> the accepted alarm decisions shall have precision >= 0.97.

> At the selected operating point, the system shall recall at least 90% of
> the critical-defect cases in the locked validation set.

Weak requirements hide the question:

> The model should be accurate.

> Confidence should be high.

The strong versions identify what will be counted and what "good enough" means.

3.3 Separate model quality from workflow capacity

Suppose a strict threshold rejects 40 percent of inspected parts. The accepted decisions may be very precise, but a human review station may not have capacity for the rejected cases. The system requirement therefore needs both quality and coverage:

```
precision among accepted defect decisions >= 0.97
and at least 70% of all parts receive an automatic decision
```

The **acceptance rate** or **coverage** is the fraction of cases not rejected. A model-policy pair can improve precision by rejecting more cases. Reporting precision without coverage can make an unusable policy look excellent.

3.4 Evaluation questions at three levels

Keep three levels separate:

1. **Model question:** How well do scores rank or classify independent examples?
2. **Policy question:** What happens after argmax, thresholds, rejection, or temporal logic?
3. **System question:** Does the complete device meet response-time, resource, and safety requirements in the intended environment?

A held-out CSV can answer many model and policy questions. It cannot establish end-to-end microphone behavior, physical latency, or field reliability by itself.

---

4. Ground truth and evaluation data

4.1 Ground truth is an engineering artifact

Ground truth is the reference answer used to judge a prediction. It may come from a measurement standard, expert annotation, a controlled experiment, or later observed outcomes. It is not automatically perfect.

For a dimension defect, calibrated metrology may provide strong ground truth. For a subtle surface defect, two inspectors may disagree. For message triage, the eventual handling team may change the label after reading context that the model never received.

Record how labels were produced:

- who or what supplied the label;
- what instructions or measurement threshold were used;
- how disagreements were resolved;
- whether the label was created before or after seeing the model output;
- which cases were excluded and why.

If the reference labels are noisy, the measured model error mixes model mistakes with label mistakes.

4.2 The test unit must be independent

One row is not always one independent test case. Ten overlapping windows from the same spoken phrase are highly related. Fifty video frames of the same part share lighting, position,

and physical identity. Randomly placing those rows into train and test sets leaks information across the boundary.

Choose a grouping unit that matches the source of variation:

- split speech by speaker, not by audio window;
- split equipment data by machine or run, not by adjacent timestamp;
- split images by physical object or capture session, not by augmented copy;
- split medical data by patient, not by image;
- split messages by thread or sender when those features can repeat.

The held-out set should contain independent groups that were not used to train the model or tune the policy.

4.3 Training, validation, and test have different jobs

- **Training data** changes model parameters.
- **Validation data** supports design choices such as architecture, preprocessing, and threshold selection.
- **Test data** is reserved for a final estimate after choices are locked.

If you repeatedly inspect test performance and revise the threshold, the test set becomes part of development. Its final score will usually be optimistic. A practical workflow is:

1. choose candidate policies on validation data;
2. write down and lock the selected policy;
3. run the test evaluation once;
4. report both the locked protocol and the result.

For coursework and early prototypes, data may be too limited for a pristine final test. The honest response is to name the limitation, not to call tuned-on data an independent test.

4.4 Representative does not mean merely large

A large evaluation set can still be unrepresentative. A warehouse alarm tested on 20,000 quiet-office recordings may say little about forklifts, reverberation, radio chatter, or different microphone placements.

Describe the deployment population and sample its important conditions:

- users, speakers, operators, or sites;
- hardware units and firmware versions;
- lighting, noise, distance, orientation, and motion;
- expected class prevalence;
- routine cases, difficult cases, and plausible non-target cases.

Subgroup reporting can reveal failures hidden by an overall average. Use subgroups that correspond to plausible mechanisms, not post-hoc categories chosen only because a result looks interesting.

4.5 Integrity checks before metrics

Before computing any metric, verify the evaluation table itself:

```
assert len(rows) > 0
assert all(row.true_label in label_order for row in rows)
assert all(len(row.scores) == len(label_order) for row in rows)
assert all(abs(sum(row.scores) - 1.0) < 1e-4 for row in rows)
assert len({row.example_id for row in rows}) == len(rows)
```

Also count examples by class and group. Metrics computed from a malformed or duplicated table can look polished and still be meaningless.

Check 3

1. Why is splitting overlapping windows at random a form of leakage?
2. Which set should be used to select a threshold? Which set should estimate final performance after the threshold is locked?
3. Give one reason that a 20,000-example test set could still be weak evidence.

---

5. Confusion matrices preserve the structure of mistakes

5.1 Build the matrix

A confusion matrix counts predicted classes against true classes. This chapter uses:

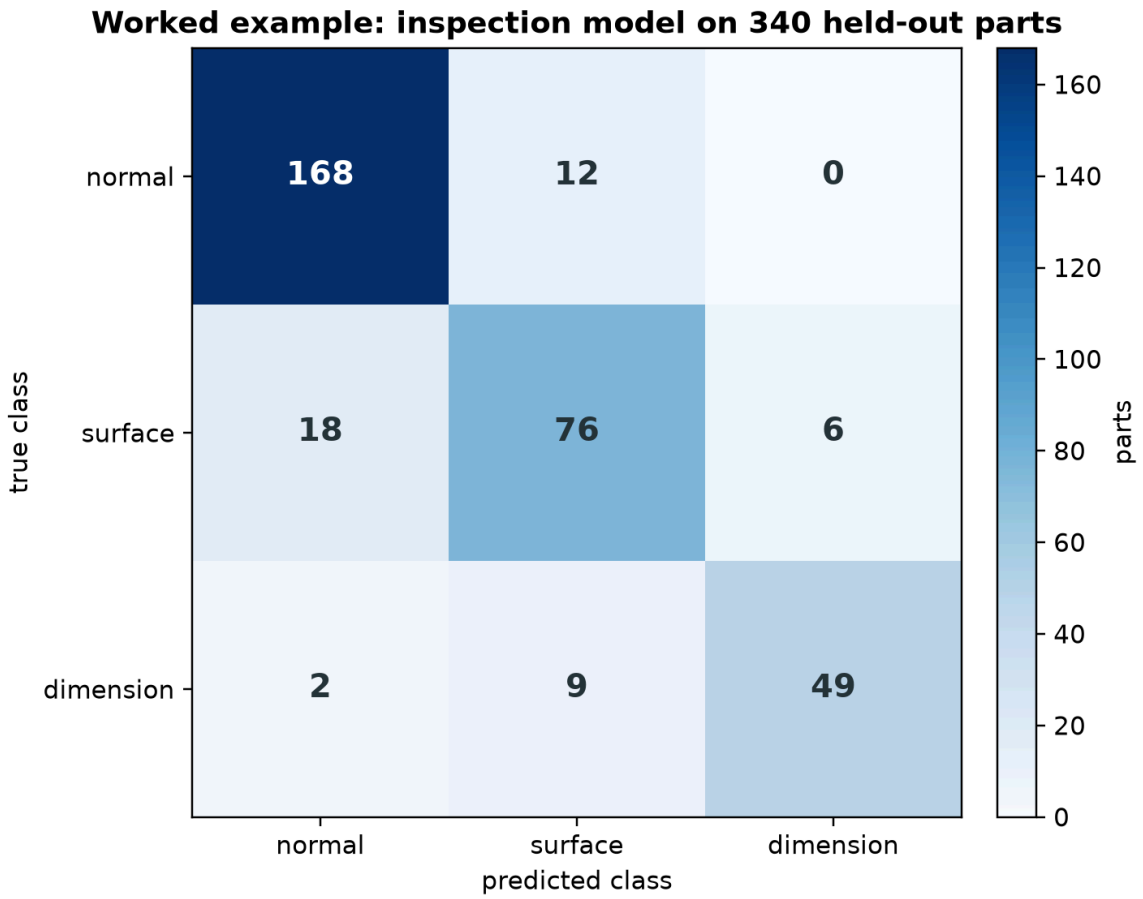
- rows for the true class;
- columns for the predicted class.

Always label the axes because some tools use the transpose.

For each evaluation example:

1. find the row for the ground-truth class;
2. find the column for the predicted class;
3. add one to that cell.

The diagonal contains correct classifications. Off-diagonal cells show specific confusions.



Confusion matrix for a three-class automated inspection example.

Figure 3. The model was evaluated on 340 held-out parts. The matrix shows not only how often it was correct, but where the errors went.

The corresponding counts are:

true class / predicted class	normal	surface defect	dimension defect	row total
normal	168	12	0	180
surface defect	18	76	6	100
dimension defect	2	9	49	60
column total	188	97	55	340

5.2 Read rows and columns differently

A row answers: **Where did examples of this true class go?**

Of 100 parts with a surface defect, 76 were classified correctly, 18 were called normal, and 6 were called dimension defects. The most consequential cell may be the 18 that were released as normal.

A column answers: **Where did predictions of this class come from?**

The `surface defect` column contains 97 predictions. Only 76 are true surface defects; 12 are normal parts and 9 have dimension defects. That column is the source for the precision calculation in the next section.

5.3 Accuracy is the diagonal divided by the total

Overall accuracy is:

```
accuracy = correct predictions / all predictions
         = (168 + 76 + 49) / 340
         = 0.862
```

Accuracy summarizes the matrix but discards its structure. It treats confusing one defect type for another as equivalent to releasing a defective part as normal. The system may not treat those errors as equivalent.

5.4 The question changes the matrix

The same three-class result can be collapsed into a binary question: **defect or normal?** If either defect label causes the part to be diverted, then confusing surface and dimension defects does not change the first-stage routing decision.

For the binary `any defect` decision:

```
TP = 76 + 6 + 9 + 49 = 140
FN = 18 + 2 = 20
FP = 12 + 0 = 12
TN = 168
```

The multiclass accuracy was 0.862, while the binary routing accuracy is `(140 + 168) / 340 = 0.906`. Neither is the one "real" accuracy. They answer different engineering questions.

> **Common misconception:** A confusion matrix is not merely a prettier accuracy table. Its purpose is to preserve which classes were confused so that errors can be connected to consequences.

Check 4

Use Figure 3.

1. How many true dimension defects were classified as normal?
2. How many predictions of `surface defect` were false?

3. Why is the binary defect-routing accuracy higher than the three-class accuracy?

---

6. Precision, recall, specificity, and F1

6.1 One class versus the rest

To evaluate one class, temporarily treat it as positive and every other class as negative:

- **true positive (TP):** the class is present and predicted;
- **false positive (FP):** the class is predicted but absent;
- **false negative (FN):** the class is present but missed;
- **true negative (TN):** the class is absent and not predicted.

For `surface defect` in Figure 3:

```
TP = 76
FN = 18 + 6 = 24
FP = 12 + 9 = 21
TN = 340 − 76 − 24 − 21 = 219
```

The row supplies `TP + FN`; the column supplies `TP + FP`.

6.2 Precision: can an accepted positive be trusted?

```
precision = TP / (TP + FP)
          = 76 / (76 + 21)
          = 0.784
```

Precision conditions on the **prediction**. Of the parts predicted as surface defects, 78.4 percent actually had that label.

High precision matters when false alarms are expensive: unnecessary shutdowns, needless human review, or actions that are difficult to reverse.

6.3 Recall: how many actual positives were found?

```
recall = TP / (TP + FN)
       = 76 / (76 + 24)
       = 0.760
```

Recall conditions on the **truth**. Of the parts that actually had surface defects, 76.0 percent were identified as such.

High recall matters when misses are expensive: an unsafe condition left undetected, a critical request ignored, or a defective part shipped.

Recall is also called sensitivity or true-positive rate.

6.4 Specificity: how many negatives were left alone?

```
specificity = TN / (TN + FP)
            = 219 / (219 + 21)
            = 0.913
```

Specificity conditions on actual negatives. It measures how well the system avoids false positives. The false-positive rate is `1 - specificity`, here `0.087`.

6.5 F1 balances precision and recall

```
F1 = 2 * precision * recall / (precision + recall)
   = 0.772
```

F1 is the harmonic mean of precision and recall. It is useful when both false positives and false negatives matter and a single comparison number is needed. It does not include true negatives, and it does not encode the actual cost of either error.

6.6 Choose the positive class from the decision

There is no universal positive class. In a fault alarm, `fault` is usually positive. In an access-control system, the risky decision might be `grant access`, even if most examples are legitimate. In a reject-defective-part workflow, `defect` is positive for detection, while `release as normal` may be the action that deserves the strongest precision requirement.

State the positive condition in words before writing a formula.

6.7 Macro, micro, and weighted averages

With many classes, reports often aggregate per-class metrics:

- **macro average:** compute the metric for each class and average classes equally;
- **weighted macro average:** weight each class by its number of examples;
- **micro average:** pool all class decisions before computing the metric.

Macro averaging exposes weak minority classes. Weighted and micro summaries can be dominated by common classes. Report per-class results when individual classes matter.

Check 5

1. For a false-alarm-sensitive system, which metric directly answers “when the system raises an alarm, how often is it right”?
2. For a miss-sensitive system, which metric directly answers “how many real faults were detected”?

3. Why can a high F1 score still be insufficient evidence for deployment?

---

7. Class imbalance and deployment prevalence

7.1 A trivial classifier can have high accuracy

Suppose only 1 percent of network events are intrusions. A classifier that always predicts `normal` has 99 percent accuracy and zero intrusion recall. Accuracy rewards the dominant class while hiding complete failure on the rare class.

A baseline should therefore be part of every evaluation. Useful baselines include:

- always predict the most common class;
- random prediction according to class prevalence;
- a simple rule based on one measured feature;
- the previous system or human workflow.

A complex model should earn its complexity by improving the metrics connected to the decision.

7.2 Precision depends on prevalence

Sensitivity and false-positive rate describe conditional behavior within positives and negatives. Precision also depends on how common positives are.

Consider a detector with:

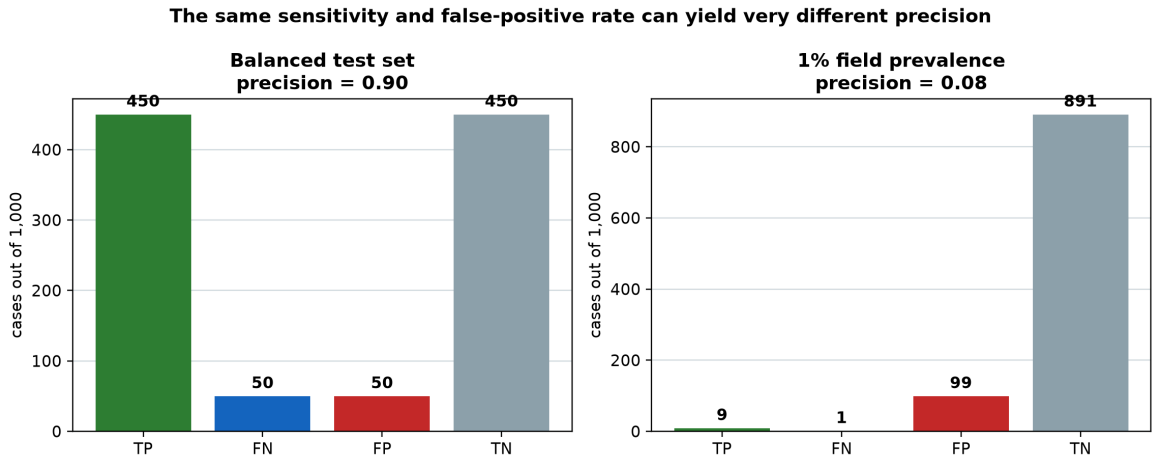
```
recall = 0.90
false-positive rate = 0.10
```

On a balanced population of 1,000 cases, there are 500 positives and 500 negatives:

```
TP = 450, FN = 50, FP = 50, TN = 450
precision = 450 / (450 + 50) = 0.90
```

If positives occur in only 1 percent of 1,000 field cases:

```
TP = 9, FN = 1, FP = 99, TN = 891
precision = 9 / (9 + 99) = 0.083
```



Class prevalence can radically change precision.

Figure 4. The detector has the same recall and false-positive rate in both scenarios. Its precision changes because the deployment base rate changes.

This is the base-rate effect. A balanced benchmark is useful for comparing behavior across classes, but its raw precision may not describe field operation.

7.3 Report counts with rates

Rates without counts hide sample size. A measured recall of `1.00` could mean 10 successes out of 10 or 10,000 out of 10,000. The second is stronger evidence.

Report both:

```
critical-fault recall = 0.94 (94 / 100 held-out faults)
```

Perfect observed performance does not prove perfect future performance. A useful rough rule is that after observing zero failures in `n` independent trials, an approximate 95 percent upper bound on the failure rate is `3 / n`. Zero failures in 30 trials is therefore consistent with a future failure rate near 10 percent. This rule is not a substitute for a full uncertainty analysis, but it prevents the most common overclaim.

7.4 Evaluation sets may intentionally differ from deployment

Balanced sets make per-class comparisons efficient. Natural-prevalence sets estimate operational workload. Both can be useful if their roles are named:

- use a balanced set to measure class-conditional recall and confusion;
- use expected field prevalence to estimate alert volume and precision;
- confirm assumptions with pilot deployment data when possible.

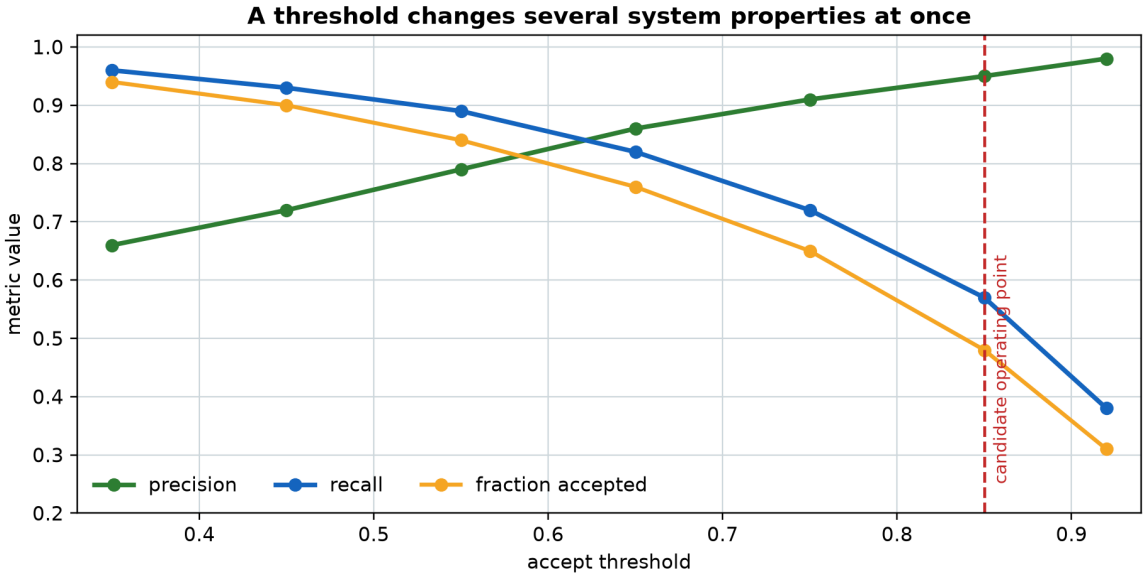
Do not quietly mix the two interpretations.

---

8. Thresholds and operating points

8.1 A threshold changes the confusion matrix

Increasing an acceptance threshold usually rejects more borderline predictions. For a target class, false positives often fall, increasing precision. True positives may also fall, decreasing recall. Coverage decreases because fewer cases receive an automatic decision.



Illustrative threshold sweep showing precision, recall, and fraction accepted.

Figure 5. Threshold selection is not "make confidence high." It is the choice of an operating point among measurable tradeoffs.

The illustrative values are:

threshold	precision	recall	accepted fraction
0.35	0.66	0.96	0.94
0.55	0.79	0.89	0.84
0.65	0.86	0.82	0.76
0.75	0.91	0.72	0.65
0.85	0.95	0.57	0.48
0.92	0.98	0.38	0.31

No row is best without a requirement. A human-review workflow may accept the 0.85 row. A critical fault detector may find its recall unacceptable.

8.2 Sweep first, select second

Given a validation table containing truth labels and score vectors:

1. define the accepted positive decision at a candidate threshold;
2. compute TP, FP, FN, and TN;
3. compute required metrics and coverage;
4. repeat for a range of thresholds;
5. select a threshold using the written requirement;
6. lock the threshold before final test evaluation.

Pseudocode:

```
for threshold in candidates:
    accepted = [
        (argmax(row.scores), max(row.scores))
        for row in validation_rows
    ]
    decisions = [
        label if score >= threshold else "reject"
        for label, score in accepted
    ]
    report(threshold, metrics(decisions, validation_truth))
```

The exact definition must handle rejected true positives and rejected negatives consistently. For an automatic positive action, a rejected positive is usually counted as a miss for recall, while a rejection is not counted as an accepted false positive.

8.3 Select the lowest threshold that meets a hard precision requirement

Suppose an automatic action requires precision at least 0.95, and recall should be as high as possible after meeting that requirement. A defensible selection rule is:

```
among thresholds whose validation precision >= 0.95,
choose the one with highest recall
```

If precision increases monotonically in the measured sweep, this is often the lowest threshold that satisfies the bound. State how ties are resolved and what happens if no candidate meets the requirement.

The requirement chooses the threshold. The visually pleasing crossing point of two curves does not.

8.4 PR and ROC curves answer related questions

A receiver operating characteristic (ROC) curve plots recall against false-positive rate. A precision-recall (PR) curve plots precision against recall. Both are created by sweeping a score threshold.

ROC views the detector's ability to separate positives from negatives. PR focuses directly on the quality and completeness of positive predictions. When the positive class is rare, PR plots often make the practical false-alarm burden easier to see.

Area under a curve summarizes ranking over many thresholds. A deployed system still needs one operating point. A high area does not choose that point or verify an operational requirement.

8.5 Thresholds can be class-specific

A multiclass system may use different thresholds when actions have different consequences. For example, auto-releasing a part as normal may require stronger evidence than routing it to a non-destructive inspection station.

Class-specific thresholds add flexibility and validation burden. Every additional policy parameter is another choice that can overfit a small validation set. Prefer the simplest policy that meets the real requirements.

Check 6

1. Why does increasing a threshold often improve precision and reduce recall?
2. Which data split should determine the threshold?
3. A threshold has precision 0.99 and coverage 0.03. What important system question remains?
4. Why does area under a curve not replace operating-point selection?

---

9. Sequential predictions require temporal policy

9.1 A stream is not a collection of independent commands

Embedded classifiers often evaluate overlapping windows. A one-second audio window might be processed every 250 ms. One physical event then appears in several consecutive windows. If each accepted window triggers an action, one event may cause four actions.

The classifier has not necessarily failed. The system has failed to define event semantics.

This is the same distinction encountered with a physical button:

- a **level** says a condition is currently present;
- an **edge** says the condition has newly become present;
- an **event** is the application meaning assigned to that transition.

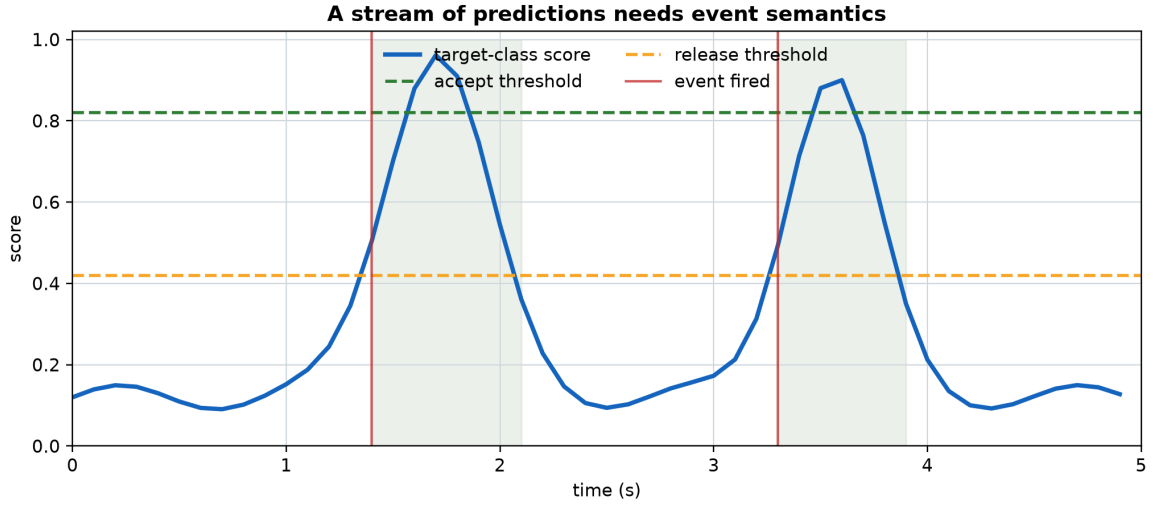
9.2 A simple latch creates one event per episode

One policy has two states, armed and latched:

```
ARMED:
  if target score crosses the accept condition:
    emit one event
    enter LATCHED

LATCHED:
  do not emit another event
  return to ARMED after the evidence has been released
```

The release condition can be a lower score threshold, a different class, a minimum quiet period, a maximum latch time, or a combination. A lower release threshold than the accept threshold creates hysteresis and reduces chatter near one boundary.



A sequential policy converts repeated high-scoring windows into discrete events.

Figure 6. Two episodes cross the accept threshold. Each produces one event, even though several windows remain above the threshold.

9.3 Debouncing, refractory periods, and voting are different tools

- **Debouncing** rejects rapid transitions that are unlikely to be distinct physical events.
- A **refractory period** blocks new events for a fixed time after one event.
- **Voting** requires agreement across several recent predictions.
- **Smoothing** averages scores or features across time.
- **Hysteresis** uses different accept and release boundaries.

These techniques solve related but distinct problems. A fixed refractory period can suppress a legitimate second event. Voting can add latency. Smoothing can erase short events. A latch without a maximum duration can remain stuck if the release condition never arrives.

Choose a policy from expected event duration, spacing, and consequences. Test traces that include:

- one isolated target episode;

- one long target episode spanning many windows;
- two target episodes separated by a short gap;
- a near-threshold non-target episode;
- a target followed by silence or another class;
- a release signal that is delayed or missing.

9.4 Evaluate the event stream, not only the windows

Window-level accuracy can be high while event behavior is poor. Event-level evaluation needs matching rules:

- what time tolerance associates a detected event with a true event;
- whether duplicate detections count as false positives;
- how two nearby true events are separated;
- how long after the event a detection is considered late.

Report event precision, event recall, duplicate count, and detection delay when the product acts on events. These metrics describe the system more directly than window accuracy.

9.5 Keep the model and policy testable in isolation

Record score traces from real inputs. Then replay those traces through the policy on a desktop computer. This separates two questions:

- did the model produce useful scores?
- did the temporal policy interpret them correctly?

Trace replay enables deterministic regression tests for one-event-per-episode behavior without repeating every physical experiment.

Check 7

1. Why can one physical event generate several accepted windows?
2. What problem does a release condition solve?
3. Name one risk introduced by majority voting or smoothing.
4. Why is event-level evaluation different from window-level evaluation?

---

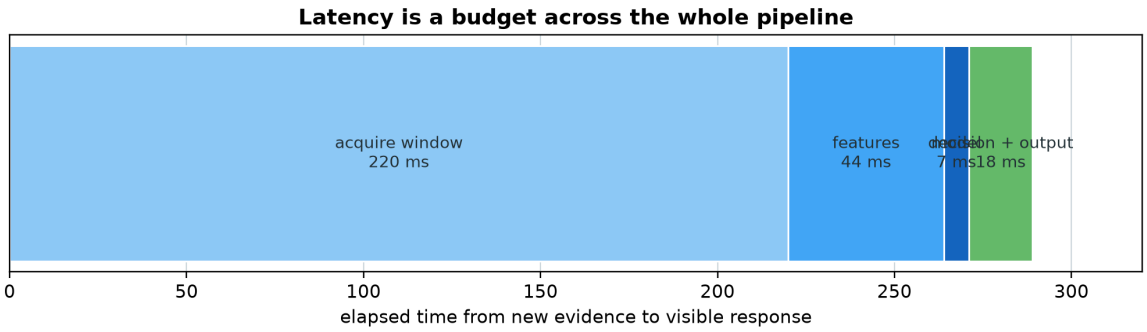
10. Latency, throughput, memory, and power

10.1 Latency has a start and an end

“Inference takes 7 ms” is incomplete. The user or physical process may wait for data acquisition, preprocessing, scheduling, policy, and output updates.

Define latency with endpoints:

```
model latency: feature tensor ready -> scores ready
decision latency: scores ready -> accepted/rejected decision ready
end-to-end latency: new physical evidence available -> observable response
```



Example decomposition of end-to-end latency.

Figure 7. A fast model can live inside a slower system. Acquisition window length may dominate the response budget.

The numbers in Figure 7 are an illustrative budget, not a hardware claim. Actual values must be measured on the target configuration.

10.2 Measure distributions, not one lucky run

Timing varies with cache state, background work, communication, and output updates. Record multiple trials and report at least:

- number of trials;
- whether warm-up runs were excluded;
- mean or median;
- a high percentile or maximum when deadlines matter;
- hardware, clock, firmware, and build configuration.

Averages hide rare slow responses. A system that must respond within 500 ms should report how often that deadline was exceeded.

10.3 Throughput is not the inverse of user-perceived latency

Throughput is the number of inputs processed per unit time. A pipeline can acquire the next input while processing the current one, so throughput and latency are related but not identical.

For overlapping windows:

```
window length = 1,000 ms
window step = 250 ms
inference time = 60 ms
```

The system produces four predictions per second because the step is 250 ms. The input window still contains one second of history. The 60 ms inference time fits within the step, leaving processing margin.

10.4 Memory claims need a definition

Embedded reports may list model size, flash usage, static RAM, heap, arena size, or peak working memory. These are not interchangeable.

Record:

- binary flash size and model-parameter size;
- static RAM reported by the build;
- peak dynamic allocation or inference arena when available;
- external memory requirements such as PSRAM;
- the exact board configuration used for the measurement.

A model that fits in flash can still fail at runtime if its working buffers do not fit in RAM.

10.5 Relative power measurements can support early comparisons

Absolute energy measurement requires calibrated instrumentation and a defined workload. Early prototypes can still compare steady states relative to a baseline:

```
relative current = measured current in state / idle current
```

Useful states might include idle, sensing, continuous inference, and inference plus display or communication. Hold each state long enough for the meter to stabilize, use the same power path, and repeat measurements. Do not mix USB sources or include a laptop’s unrelated load.

Relative measurements answer comparison questions such as “continuous inference draws about 1.8 times the idle current under this setup.” They do not establish battery life without duty cycle, voltage, regulator efficiency, and battery capacity.

---

11. Worked case: automated quality screening

This case connects requirements, data, metrics, threshold selection, and system costs. It uses the inspection model from Figure 3.

11.1 System description

A camera observes each molded part. The model scores three classes:

```
normal
surface_defect
dimension_defect
```

The first-stage policy has three actions:

- release a confidently normal part;
- divert a confidently defective part to the appropriate inspection lane;
- send an uncertain part to manual review.

Shipping a defective part is more costly than manually reviewing a normal part. The manual station can handle no more than 35 percent of production.

11.2 Translate the workflow into measurable requirements

One possible set of requirements is:

```
R1: Recall for any defect >= 0.92 on held-out production sessions.
R2: Precision of automatic normal release >= 0.99.
R3: Automatic coverage >= 0.65 so manual review remains within capacity.
R4: 99th-percentile end-to-end decision latency <= the conveyor time budget.
```

Notice that R2 treats `normal release` as the positive action because that is the action whose false positives are defective parts being shipped. Metric vocabulary must follow the decision, not a memorized convention that "defect is always positive."

11.3 Design the evaluation set

The evaluation unit is a physical part, not an image. Multiple images from the same part remain in one split. Capture sessions are held out so test images include new lighting and fixture variation.

Ground truth combines calibrated dimensional measurement and a two-inspector surface review. Disagreements are adjudicated without showing model output. The manifest records part ID, batch, capture session, hardware unit, true labels, and image path.

The validation set supports threshold selection. A separate locked test set is used after the policy is selected.

11.4 Start with the confusion matrix, not a favorite metric

Figure 3 immediately shows that 20 defects were classified as normal under argmax. It also shows 12 normal parts classified as defects. The matrix suggests two policy questions:

- can a rejection threshold reduce defective releases enough to satisfy R2?
- can it do so without rejecting more than 35 percent of all parts?

The matrix does not answer those questions because argmax discards score magnitude. The threshold sweep does.

11.5 Select and lock a policy

For each candidate pair of class-specific thresholds, compute:

- any-defect recall;
- normal-release precision;
- coverage;
- manual-review rate;
- counts behind every metric.

Discard policies that fail R1, R2, or R3. If several remain, choose the simplest or the one with the largest margin to the most important requirement. Record the selected thresholds and a hash or version identifier for the model and validation table.

Then run the locked test once. If the test fails a requirement, report the failure. Returning to validation and redesigning is legitimate, but the next test result belongs to a new development cycle.

11.6 Evaluate the deployed path

Replay evaluation verifies policy logic. Hardware tests verify camera acquisition, preprocessing, model execution, output routing, and latency. Pilot production verifies lighting, contamination, operator behavior, and field prevalence.

The final claim should remain bounded:

> On 1,200 held-out parts from four production sessions, the locked policy met the stated defect-recall, normal-release precision, and coverage requirements. It was also tested on the target device for end-to-end latency. Performance on other cameras, materials, and lighting conditions was not established.

That claim is useful because it says exactly what the evidence covers.

---

12. Designing a defensible evaluation protocol

12.1 Protocol template

Before running the evaluation, write one page that answers:

1. **Purpose:** What decision will this result support?
2. **System boundary:** Model only, policy replay, or full hardware path?
3. **Population:** Which future inputs should the data represent?

4. **Unit and grouping:** What is one independent example, and how are related examples kept together?
5. **Ground truth:** How are labels produced and checked?
6. **Data roles:** Which data tune the system, and which remain locked?
7. **Decision rule:** Label order, argmax, thresholds, rejection, and temporal policy.
8. **Metrics:** Which rates, counts, subgroups, coverage, and resource measures will be reported?
9. **Acceptance criteria:** What must be true to pass?
10. **Artifacts:** Which files allow another engineer to reproduce the result?

12.2 Reproducibility artifacts

A compact evidence package often includes:

- data manifest with stable example IDs and group IDs;
- label order and model identifier;
- raw score table, not only final labels;
- evaluation script or notebook;
- threshold-sweep table;
- selected-policy configuration;
- confusion matrix and metric table with counts;
- sequential traces and expected events, when applicable;
- hardware, firmware, and build configuration;
- timing or power logs;
- short limitations statement.

Raw scores preserve future options. If requirements change, thresholds can be reevaluated without rerunning the model.

12.3 Claim discipline

Match verbs to evidence:

- **observed:** directly seen in a small test;
- **measured:** produced by a defined protocol and instrument;
- **estimated:** inferred from a sample with stated assumptions;
- **verified:** shown to meet a written requirement under stated conditions;
- **validated:** demonstrated to satisfy an intended use with representative evidence.

Avoid “proved reliable” from a short demonstration. Avoid “real-time” without a deadline and measured latency. Avoid “95 percent confident” unless a statistical interval was actually computed.

12.4 Failure is a result

An evaluation that finds a requirement violation has succeeded as an evaluation. Preserve the failing cases and decide whether to:

- improve data or preprocessing;
- change the model;
- change the policy or fallback;
- narrow the intended operating conditions;
- revise an unrealistic requirement through an explicit engineering decision.

Do not quietly remove difficult cases because they reduce a score. Exclusions require a reason connected to the stated population.

12.5 Final review questions

Before accepting a report, ask:

- Is the model version unambiguous?
- Are labels and score columns in the same order?
- Are train, validation, and test groups disjoint?
- Does every metric include a denominator or count?
- Was the threshold chosen before the final test result was examined?
- Are rejected cases included consistently?
- Does sequential testing check duplicates and missed events?
- Were hardware claims measured on the stated target?
- Are limitations visible next to the claim they constrain?

End-of-chapter problems

Problem 1: Score interpretation

A four-class model returns `[0.27, 0.25, 0.26, 0.22]`.

1. What is the argmax class index?
2. What does argmax fail to communicate?
3. Under an acceptance threshold of 0.70, what is the policy result?

Problem 2: Confusion matrix construction

Build a confusion matrix with rows=true and columns=predicted for:

```
truth: A A A B B B C C
pred:  A B A B C B C A
```

Compute overall accuracy.

Problem 3: One-vs-rest metrics

Using your matrix from Problem 2, treat class `B` as positive. Compute TP, FP, FN, TN, precision, recall, and specificity.

Problem 4: Reframe the decision

In the inspection matrix from Figure 3, treat "automatic release as normal" as the positive action. Define TP, FP, FN, and TN in words. Which error corresponds to shipping a defective part?

Problem 5: Threshold selection

A validation sweep gives:

threshold	precision	recall	coverage
0.60	0.91	0.88	0.86
0.70	0.95	0.81	0.74
0.80	0.98	0.69	0.58
0.90	1.00	0.41	0.25

The requirement is precision at least 0.97 and coverage at least 0.50. Select the feasible row with highest recall. Explain why the 0.90 row is not automatically preferable.

Problem 6: Leakage

A dataset contains 40 recordings from each of 20 speakers. A student randomly splits the 800 files into train and test. Explain the leakage risk and propose a better split.

Problem 7: Base rates

A detector has recall 0.95 and false-positive rate 0.02. In 10,000 field cases, 100 are actually positive. Compute expected TP, FN, FP, TN, and precision.

Problem 8: Sequential policy

A classifier runs every 200 ms on an 800 ms window. One physical event is present for one second. Explain why acting on every accepted window can duplicate the action. Propose a state-based policy and list two traces needed to test it.

Problem 9: Latency

A system reports 12 ms model inference time, but uses a 1,000 ms non-overlapping acquisition window and updates its display every 250 ms. Explain why "12 ms response time" is not a valid end-to-end claim. Identify useful latency endpoints.

Problem 10: Evaluation claim

Rewrite the following claim so it matches limited evidence:

> Our classifier is 100 percent reliable in the real world.

The available evidence is 50 correct decisions on held-out examples from one capture session and one hardware unit.

Problem 11: Protocol design

Design a one-page protocol for an access-request classifier. False grants are more costly than false denials, and rejected requests can be reviewed by a person. Include the population, independent unit, data split, ground truth, metrics, coverage constraint, threshold-selection rule, and final claim format.

Problem 12: Critique a report

A report states: "We selected threshold 0.82 because it had the best test precision. The test set contained augmented crops from the same images used in training. Accuracy was 98 percent, so the system is production ready." Identify at least four problems and state how to correct each one.

Selected responses to checks and problems

Check 1

1. Examples include sample rate, tensor shape, units, color order, normalization, and label order.
2. The predicted label is `approve`; the accepted decision is reject; the action depends on the defined fallback and may be no automatic action.
3. Their thresholds, rejection rules, input populations, or temporal policies may differ.

Check 2

1. Argmax is defined to select the largest score even when all scores are close.
2. Unknown is a learned class; rejection is a post-model policy result.
3. A reliability evaluation showing that predictions near 0.80 are correct about 80 percent of the time on representative held-out data.

Check 3

1. Related windows share the same underlying event, source, and conditions, allowing near copies into both sets.
2. Select on validation; estimate final performance on the locked test set.
3. It may omit important users or conditions, use weak labels, contain duplicates, or differ from deployment prevalence.

Check 4

1. Two.
2. Twenty-one: 12 normal parts plus 9 dimension-defect parts.
3. Confusing one defect type for the other is wrong for three-class labeling but still correct for defect-versus-normal routing.

Check 5

1. Precision.
2. Recall.
3. F1 omits true negatives, does not encode actual error costs, and may come from unrepresentative or too-small data.

Check 6

1. Borderline false positives are rejected, but some true positives are rejected as well.
2. Validation data.
3. Whether rejecting 97 percent of cases is operationally acceptable.
4. It summarizes ranking across thresholds but does not identify the deployed decision rule.

Check 7

1. Overlapping windows contain the same physical event.
2. It determines when the system may emit another event.
3. Either can add delay or erase a short legitimate event.
4. Event evaluation must match detections to physical episodes and count duplicates and delay, not merely classify individual windows.

Problem 2

```
        predicted
true    A  B  C
A       2  1  0
B       0  2  1
C       1  0  1
```

Accuracy is `(2 + 2 + 1) / 8 = 0.625`.

Problem 3

For class B: `TP=2`, `FP=1`, `FN=1`, `TN=4`. Precision and recall are both `2/3`; specificity is `4/5`.

Problem 5

Threshold 0.80 is the only row that meets both constraints and therefore has the highest recall among feasible rows. Threshold 0.90 has higher precision but fails the coverage constraint and sacrifices recall without a stated benefit.

Problem 7

Expected counts are `TP=95`, `FN=5`, `FP=198`, and `TN=9,702`. Precision is `95 / (95 + 198) = 0.324`. A low false-positive rate can still produce more false alarms than true detections when positives are rare.

Problem 10

One bounded rewrite is:

> The classifier correctly handled all 50 held-out examples from one capture session on one hardware unit. The sample does not establish performance across other sessions, devices, users, or operating conditions.

References and further reading

1. P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," arXiv:1804.03209, 2018. https://arxiv.org/abs/1804.03209
2. T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, 27(8), 2006. https://doi.org/10.1016/j.patrec.2005.10.010
3. J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," ICML, 2006. https://doi.org/10.1145/1143844.1143874
4. T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLOS ONE*, 2015. https://doi.org/10.1371/journal.pone.0118432
5. A. P. Bradley, "The Use of the Area Under the ROC Curve in the Evaluation of Machine Learning Algorithms," *Pattern Recognition*, 30(7), 1997. https://doi.org/10.1016/S0031-3203(96)00142-2

6. C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," ICML, 2017. https://proceedings.mlr.press/v70/guo17a.html
7. D. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," NeurIPS, 2015. https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems